XYCConcentrate GrowLiquidity: How It Works, What Is Broken, and How to Fix It

Abstract

This document explains the `XYCConcentrate` GrowLiquidity mechanism used in the 1inch SwapVM, starting from first principles with worked numerical examples. We then identify two critical bugs found during the Bail Security audit—swapped delta formulas and a fundamentally broken 3D/4D generalization—and show exactly how to fix each one. Finally, we analyze the additivity (path-independence) properties under different fee configurations.

Contents

I How the Mechanism Works **2**

1 The Idea: Concentrated Liquidity on a Constant-Product Curve **2**

2 The Standard Math (Uniswap V3 Style) **2**

3 How `XYCConcentrate` Works in Practice **3**

4 How `computeDeltas` Constructs the Pool **4**

II What Is Broken and How to Fix It **4**

5 Bug 1: The Delta Formulas Are Swapped **4**
5.1 The correct formulas . . . 5
5.2 What the implementation does . . . 5
5.3 When the bug is invisible . . . 5
5.4 Numerical comparison . . . 5
5.5 The fix . . . 5

6 Design Note: `computeDeltas` Takes Price as Input **6**

7 Note on Stored L (2D Is Fine; >2D Is Not) **6**

8 Bug 2: 3D/4D Pairwise Decomposition Is Fundamentally Broken **7**
8.1 The fix . . . 7

III Additivity Analysis (2D) **7**

9 Additivity: What It Means and Why It Matters **8**

10 Fee-Free: Strictly Additive **8**

11 Input-Fee: Subadditive (Safe) **9**

12 Output-Fee: Superadditive (Exploitable) **9**

IV Summary **10**

13 All Issues and Fixes at a Glance **10**

14 Additivity Summary **10**

Part I
How the Mechanism Works

1 The Idea: Concentrated Liquidity on a Constant-Product Curve

A plain constant-product AMM holds two tokens with reserves (x, y) satisfying $xy = k$. Liquidity is spread across all prices from 0 to ∞, which is capital-inefficient.

Concentrated liquidity restricts trading to a chosen price range $[P_{\text{lo}}, P_{\text{hi}}]$. The trick is to add *virtual offsets* ("deltas") to the real balances so that the constant-product formula acts on larger *virtual reserves*:

$$X = b_x + \delta_x, \qquad Y = b_y + \delta_y,$$

where b_x, b_y are the real token balances and δ_x, δ_y are the offsets. Swaps use the virtual reserves (X, Y), but the pool only actually holds (b_x, b_y).

Example 1 (Why this helps)**.** Suppose a maker deploys $b_x = b_y = 1{,}000$ tokens at price 1 and concentrates between $P_{\text{lo}} = 0.5$ and $P_{\text{hi}} = 2.0$.

Without concentration the pool would be $(1000, 1000)$ with $k = 10^6$.

With concentration the virtual reserves are much larger (say $X = Y = 3{,}000$), giving $k = 9 \times 10^6$—nine times the effective liquidity. The trade $\Delta = 100$ of token X yields:

- Without concentration: out $= 100 \times 1000/1100 \approx 90.9$.
- With concentration: out $= 100 \times 3000/3100 \approx 96.8$.

The concentrated pool gives a better price because liquidity is deeper around the current price.

2 The Standard Math (Uniswap V3 Style)

Define the *spot price* $P = Y/X$ and *liquidity* $L = \sqrt{XY}$. Then:

$$X = \frac{L}{\sqrt{P}}, \qquad Y = L\sqrt{P}, \tag{1}$$

$$\delta_x = \frac{L}{\sqrt{P_{\text{hi}}}}, \qquad \delta_y = L\sqrt{P_{\text{lo}}}, \tag{2}$$

$$b_x = X - \delta_x = L\left(\frac{1}{\sqrt{P}} - \frac{1}{\sqrt{P_{\text{hi}}}}\right), \qquad b_y = Y - \delta_y = L\left(\sqrt{P} - \sqrt{P_{\text{lo}}}\right). \tag{3}$$

Remark 1 (Boundary conditions)**.** At $P = P_{\text{lo}}$: $b_y = 0$ (all capital in token X). At $P = P_{\text{hi}}$: $b_x = 0$ (all capital in token Y).

Example 2 (Computing deltas from scratch)**.** Take $P_{\text{lo}} = 0.25$, $P_{\text{hi}} = 4.0$, $P = 1.0$, $L = 1000$.

$$\begin{aligned} X &= 1000/\sqrt{1} = 1000, & Y &= 1000 \times \sqrt{1} = 1000, \\ \delta_x &= 1000/\sqrt{4} = 500, & \delta_y &= 1000 \times \sqrt{0.25} = 500, \\ b_x &= 1000 - 500 = 500, & b_y &= 1000 - 500 = 500. \end{aligned}$$

So a maker depositing $b_x = b_y = 500$ real tokens gets virtual reserves of $(1000, 1000)$—double the effective liquidity.

3 How `XYCConcentrate` Works in Practice

The implementation stores three pieces of data per token pair: δ_x, δ_y, L_0 (the initial liquidity). On every swap it runs these steps:

Step 1 Read the current liquidity L_{cur}. For 2D this is $L_{\text{cur}} = \sqrt{K_{\text{stored}}}$, where K was saved from the previous swap.

Step 2 Build concentrated balances. Scale the deltas proportionally to how much L has grown:

$$X = b_x + \delta_x \cdot \frac{L_{\text{cur}}}{L_0}, \qquad Y = b_y + \delta_y \cdot \frac{L_{\text{cur}}}{L_0}. \tag{4}$$

Step 3 Execute a constant-product swap on (X, Y):

$$\text{amtOut} = \frac{\Delta_{\text{eff}} \cdot Y}{X + \Delta_{\text{eff}}}, \tag{5}$$

where Δ_{eff} is the input after any fee.

Step 4 Update and store L.

$$K_{\text{new}} = (X + \text{amtIn}) \cdot (Y - \text{amtOut}), \quad L_{\text{new}} = \sqrt{K_{\text{new}}}. \tag{6}$$

Example 3 (A full swap walkthrough)**.** Start with $b_x = b_y = 500$, $\delta_x = \delta_y = 500$, $L_0 = 1000$, and $L_{\text{cur}} = L_0 = 1000$ (first swap, no growth yet). No fee ($f = 0$). Trader sends $\Delta = 100$ of token X.

Step 2. $X = 500 + 500 \times 1000/1000 = 1000$, $\quad Y = 500 + 500 \times 1000/1000 = 1000$.

Step 3. $\text{amtOut} = 100 \times 1000/1100 \approx 90.91$.

Step 4. $K = 1100 \times 909.09 = 1{,}000{,}000$ (unchanged—no fee), $\quad L = 1000$ (unchanged).

Trader receives ≈ 90.91 of token Y.

Example 4 (Swap with input-fee)**.** Same pool as above. Fee $f = 1\%$. Trader sends $\Delta = 100$.

Step 2. Same: $(X, Y) = (1000, 1000)$.

Step 3. Effective input $\Delta_{\text{eff}} = 0.99 \times 100 = 99$. amtOut $= 99 \times 1000/1099 \approx 90.08$.

Step 4. $X' = 1000 + 100 = 1100$ (full Δ credited). $Y' = 1000 - 90.08 = 909.92$. $K = 1100 \times 909.92 = 1{,}000{,}912 > 10^6$. $L = \sqrt{1{,}000{,}912} \approx 1000.46$.

The product *grew* because the fee (1 token) stayed in the pool. Next swap, the deltas scale up by $1000.46/1000 = 1.00046$, making the pool slightly deeper.

4 How `computeDeltas` Constructs the Pool

The function `computeDeltas(balanceA, balanceB, price, priceMin, priceMax)` computes the initial deltas and liquidity. Its formulas are:

$$\texttt{sqrtMin} = \sqrt{\frac{P}{P_{\text{lo}}}}, \qquad \texttt{sqrtMax} = \sqrt{\frac{P_{\text{hi}}}{P}}, \tag{7}$$

$$\texttt{deltaA} = \frac{b_x}{\texttt{sqrtMin} - 1}, \qquad \texttt{deltaB} = \frac{b_y}{\texttt{sqrtMax} - 1}, \tag{8}$$

$$\texttt{liquidity} = \sqrt{(b_x + \texttt{deltaA})(b_y + \texttt{deltaB})}. \tag{9}$$

Example 5 (Running `computeDeltas`)**.** Inputs: $b_x = b_y = 1000$, $P = 1.0$, $P_{\text{lo}} = 0.25$, $P_{\text{hi}} = 4.0$.

$$\texttt{sqrtMin} = \sqrt{1.0/0.25} = 2.0, \qquad \texttt{sqrtMax} = \sqrt{4.0/1.0} = 2.0,$$
$$\texttt{deltaA} = 1000/(2-1) = 1000, \qquad \texttt{deltaB} = 1000/(2-1) = 1000,$$
$$\texttt{liquidity} = \sqrt{2000 \times 2000} = 2000.$$

Virtual reserves: $(X, Y) = (2000, 2000)$.

Example 6 (Away from the geometric mean)**.** Same bounds, but now $P = 0.5$ (not the geometric mean $\sqrt{0.25 \times 4} = 1$):

$$\texttt{sqrtMin} = \sqrt{0.5/0.25} = \sqrt{2} \approx 1.414, \qquad \texttt{sqrtMax} = \sqrt{4.0/0.5} = \sqrt{8} \approx 2.828,$$
$$\texttt{deltaA} = 1000/0.414 \approx 2414, \qquad \texttt{deltaB} = 1000/1.828 \approx 547.$$

We will see in §5 that these values are *wrong*—the formulas for A and B are swapped.

Part II
What Is Broken and How to Fix It

5 Bug 1: The Delta Formulas Are Swapped

Audit references:

- **[H]** *computeDeltas does not obey price limits*
- **[M]** *computeDeltas — ratio of square roots of prices is not meaningful*
- **[H]** *computeDeltas — inconsistent rounding direction*

5.1 The correct formulas

From the standard math (§2), expressing the offsets in terms of real balances:

$$\delta_x = \frac{b_x}{\sqrt{P_{\text{hi}}/P} - 1} \quad \big(= b_x \sqrt{P} \,/\, (\sqrt{P_{\text{hi}}} - \sqrt{P})\big), \tag{10}$$

$$\delta_y = \frac{b_y}{\sqrt{P/P_{\text{lo}}} - 1} \quad \big(= b_y \sqrt{P_{\text{lo}}} \,/\, (\sqrt{P} - \sqrt{P_{\text{lo}}})\big). \tag{11}$$

5.2 What the implementation does

	Correct	**Implementation**
δ_x (tokenA)	$\dfrac{b_x}{\sqrt{P_{\text{hi}}/P} - 1}$	$\dfrac{b_x}{\sqrt{P/P_{\text{lo}}} - 1}$
δ_y (tokenB)	$\dfrac{b_y}{\sqrt{P/P_{\text{lo}}} - 1}$	$\dfrac{b_y}{\sqrt{P_{\text{hi}}/P} - 1}$

[BUG] The sqrt ratio arguments are swapped between tokenA and tokenB.

5.3 When the bug is invisible

The two formulas coincide when $\sqrt{P_{\text{hi}}/P} = \sqrt{P/P_{\text{lo}}}$, i.e. $P = \sqrt{P_{\text{lo}} \cdot P_{\text{hi}}}$ (the geometric mean of the bounds). This is why tests at symmetric prices may not catch the bug.

5.4 Numerical comparison

Example 7 (Correct vs. implementation at $P = 0.5$)**.** $b_x = b_y = 1000$, $P_{\text{lo}} = 0.25$, $P_{\text{hi}} = 4.0$, $P = 0.5$.

Correct:

$$\delta_x = \frac{1000}{\sqrt{4/0.5} - 1} = \frac{1000}{\sqrt{8} - 1} = \frac{1000}{1.828} \approx 547, \qquad \delta_y = \frac{1000}{\sqrt{0.5/0.25} - 1} = \frac{1000}{\sqrt{2} - 1} = \frac{1000}{0.414} \approx 2414.$$

Virtual reserves: $X = 1547$, $Y = 3414$. Check: $Y/X = 3414/1547 \approx 2.21$, and $\sqrt{P} = \sqrt{0.5} \approx 0.707$; $L = \sqrt{XY} = \sqrt{5{,}281{,}158} \approx 2298$. Verify b_y: $L(\sqrt{P} - \sqrt{P_{\text{lo}}}) = 2298 \times (0.707 - 0.5) = 2298 \times 0.207 \approx 476$.

Hmm, this doesn't match $b_y = 1000$. That is because at $P = 0.5$ with these bounds and $b_x = b_y = 1000$, the system is over-constrained—P is not a free parameter.

Implementation (swapped): `deltaA` $= 2414$, `deltaB` $= 547$.

The deltas are *exactly exchanged*, giving virtual reserves $X = 3414$, $Y = 1547$—mirror image of the correct values.

5.5 The fix

[FIX] Swap the two sqrt expressions in `computeDeltas`:

$$\texttt{sqrtForA} = \sqrt{P_{\text{hi}}/P} \qquad (\text{was: } \sqrt{P/P_{\text{lo}}}),$$

$$\texttt{sqrtForB} = \sqrt{P/P_{\text{lo}}} \qquad (\text{was: } \sqrt{P_{\text{hi}}/P}).$$

6 Design Note: `computeDeltas` Takes Price as Input

Audit reference:

- **[H]** *computeDeltas is overspecified*

`computeDeltas` takes **5 inputs**: $b_x, b_y, P, P_{\text{lo}}, P_{\text{hi}}$. In pure concentrated-liquidity math, only 4 are independent—given $(b_x, b_y, P_{\text{lo}}, P_{\text{hi}})$ the spot price P is uniquely determined.

However, this is **not a bug**. The maker calls `computeDeltas` off-chain to construct their own position. They control all 5 inputs and deliberately pass the price they believe reflects the current market. The function is a `pure` helper: no one else can call it with a "wrong" P to harm the maker.

If the maker's balances don't perfectly match the claimed price at those bounds, the resulting deltas produce slightly different concentration characteristics than the textbook formulas would imply. This is a capital-efficiency trade-off the maker accepts, not a security vulnerability.

Remark 2 (Deployment-timing risk)**.** There is a real operational risk: the market price can move between when the maker calls `computeDeltas` off-chain and when the strategy goes live and the first swap executes on-chain. If the price has moved, the pool opens at a stale price. Arbitrageurs will trade against the pool until it converges to the market price, extracting value from the maker.

Concentrated liquidity *amplifies* this effect: higher capital efficiency means more loss per unit of price movement compared to a full-range pool.

This is an inherent deployment-timing risk for any concentrated-liquidity AMM (including Uniswap V3 positions). It is not specific to the 5-parameter interface, since deriving P from $(b_x, b_y, P_{\text{lo}}, P_{\text{hi}})$ would still bake in a price that becomes stale after deployment. The mitigation is for makers to deploy when they can execute quickly and to choose wider ranges if they expect delays.

Remark 3 (Possible improvement)**.** The function could also return the *implied* P from the quadratic

$$b_x\sqrt{P_{\text{hi}}}\,(\sqrt{P})^2 - \left(b_y - b_x\sqrt{P_{\text{lo}}P_{\text{hi}}}\right)\sqrt{P} - b_y\sqrt{P_{\text{lo}}} = 0,$$

so the maker can compare it against the price they passed and gauge how much the position diverges from the textbook formulation.

7 Note on Stored L (2D Is Fine; >2D Is Not)

Audit references:

- **[H]** *GrowLiquidityXD grows liquidity for unrelated tokens*
- **[H]** *Liquidity calculation is incorrect in Fee → XYCConcentrate sequence*
- **[H]** *Liquidity will be disrupted if combined with Decay*
- **[H]** *XYCConcentrate → Fee does not work for protocol fee scenario*

The current code stores $K = X'Y'$ after each swap and derives $L = \sqrt{K}$ for the next swap.

For 2D this is correct. The strategy is immutable (the maker cannot deposit or withdraw between swaps) and Aqua tracks the real balances. Because no external changes can occur between swaps, the stored K faithfully reflects the post-swap state and correctly drives the delta scaling on the next swap. With input fees K grows monotonically, and the stored value captures that growth accurately.

For >2 tokens, storing L is still problematic (but this is subsumed by Bug 2 below). A swap A→B changes b_B, which affects the B↔C implied price and L. The pairwise decomposition stores a separate L per pair, so the A→B swap does not update the B↔C state. This is one more reason the 3D/4D approach is unsound.

8 Bug 2: 3D/4D Pairwise Decomposition Is Fundamentally Broken

Audit references:

- **[H]** *GrowLiquidityXD decomposition into pairwise AMMs is unsound for >2 assets*
- **[H]** *GrowLiquidityXD prices don't adjust for uninvolved tokens (self-arbitrage)*
- **[H]** *GrowLiquidityXD grows liquidity for unrelated tokens*
- **[M]** *computeDeltas is not meaningful for >2 assets*

The 3D and 4D variants treat each token pair as an independent 2D pool. This is **unsound**.

Theorem 1 (Impossibility of Pairwise Decomposition)**.** *For $n \geq 3$ tokens, a concentrated-liquidity constant-product AMM cannot be decomposed into $\binom{n}{2}$ independent pairwise AMMs while preserving price consistency.*

Proof sketch. For three tokens A, B, C, price consistency requires $P_{A/C} = P_{A/B} \cdot P_{B/C}$. A swap $A \to B$ changes $P_{A/B}$ but leaves the independent B/C and A/C pools untouched, so $P_{B/C}$ stays fixed. Now $P'_{A/B} \cdot P_{B/C} \neq P_{A/C}$ (which hasn't changed either). This violates the consistency condition and creates triangular arbitrage: $A \to B \to C \to A$ can extract value, and repeating it drains the pool. □

Example 8 (Triangular arbitrage (simplified))**.** Three tokens, all initially priced 1:1.

1. Swap a large amount of A for B. Now $P_{A/B}$ drops (A cheaper, B expensive).
2. The B/C pool still thinks B is worth 1 unit of C. Swap $B \to C$ at the stale (too-high) price for B.
3. The A/C pool still thinks A is worth 1 unit of C. Swap $C \to A$ at the stale price.
4. The attacker ends up with more A than they started with.

Repeat until the pool is drained.

Remark 4 (PoC exploit)**.** The Bail Security audit includes a Foundry invariant test that finds draining swap sequences on a 3-token XYCConcentrate pool.

8.1 The fix

[FIX] Disable the 3D and 4D variants. A correct n-asset concentrated-liquidity CFAMM requires:

- A single invariant over all n virtual balances (not pairwise).
- Recomputing L on each swap via $\sum_i \ln(p_i + b_i/L) = 0$ (no closed form for $n \geq 3$; requires Newton iteration).
- Careful rounding in the direction disadvantageous to the taker.

This is substantially more complex than the current approach.

Part III
Additivity Analysis (2D)

9 Additivity: What It Means and Why It Matters

Definition 1 (Additivity Types)**.** Let $\mathrm{Out}(\Delta; s)$ be the trader output for input Δ from state s, and let $F_\Delta(s)$ be the post-swap state. Define $\mathrm{Out}_{\mathrm{split}}(a, b; s) := \mathrm{Out}(a; s) + \mathrm{Out}(b; F_a(s))$.

(a) **Strictly additive:** $\mathrm{Out}(a + b; s) = \mathrm{Out}_{\mathrm{split}}(a, b; s)$ for all $a, b \geq 0$.

(b) **Subadditive:** $\mathrm{Out}(a + b; s) > \mathrm{Out}_{\mathrm{split}}(a, b; s)$ for all $a, b > 0$.

(c) **Superadditive:** $\mathrm{Out}(a + b; s) < \mathrm{Out}_{\mathrm{split}}(a, b; s)$ for all $a, b > 0$.

Why this matters for a DEX:

- **Subadditive = safe.** One-shot is better for the trader. Splitting hurts. No MEV exploit from chunking.
- **Superadditive = dangerous.** Splitting is better. MEV bots chunk trades to extract value.
- **Strictly additive = ideal.** Path-independent. No gaming in either direction.

Remark 5. The additivity type depends on the *fee placement*, not on the delta computation. So the delta bugs from Part II do not change these results.

10 Fee-Free: Strictly Additive

Theorem 2. *Without fees, the 2D GrowLiquidity mechanism is strictly additive.*

Proof. Fix virtual reserves (X, Y). The fee-free swap preserves $K = XY$:

$$(X + \Delta)(Y - \mathrm{out}) = XY.$$

One-shot $\Delta = a + b$: $\mathrm{out}_{\mathrm{one}} = (a + b)Y/(X + a + b)$.

Split a then b:

$$\mathrm{out}_1 = \frac{aY}{X + a}, \qquad X_1 = X + a, \qquad Y_1 = \frac{XY}{X + a}.$$
$$\mathrm{out}_2 = \frac{bY_1}{X_1 + b} = \frac{bXY}{(X + a)(X + a + b)}.$$

Sum:

$$\mathrm{out}_1 + \mathrm{out}_2 = \frac{aY}{X + a} + \frac{bXY}{(X + a)(X + a + b)} = \frac{(a + b)Y}{X + a + b} = \mathrm{out}_{\mathrm{one}}.$$

Since K is unchanged after each leg, L doesn't change, and deltas scale identically, confirming strict additivity. □

Example 9. $(X, Y) = (1000, 1000)$.

One-shot 200: out $= 200{,}000/1200 \approx 166.67$.

Split 100+100: $\mathrm{out}_1 = 100{,}000/1100 \approx 90.91$; state $(1100, 909.09)$; $\mathrm{out}_2 = 90{,}909/1200 \approx 75.76$; total ≈ 166.67. Equal. ✓

11 Input-Fee: Subadditive (Safe)

Definition 2. Fee rate $f \in (0,1)$. Effective input $\Delta_{\text{eff}} = (1-f)\Delta$. Full Δ is credited to the input reserve; only Δ_{eff} enters the swap.

Theorem 3. *With input-fee, GrowLiquidity is subadditive:* $\text{Out}(a+b) > \text{Out}(a) + \text{Out}(b; F_a)$ *for* $a, b > 0$.

Proof. **One-shot:** $\text{Out}_{\text{one}} = (1-f)(a+b)Y/[X + (1-f)(a+b)]$.

Split: $\text{out}_1 = (1-f)aY/[X+(1-f)a]$. Post-swap: $X_1 = X+a$, $Y_1 = XY/[X+(1-f)a]$. Since $X + a > X + (1-f)a$, the product $K_1 = X_1Y_1 > XY$ grows. $\text{out}_2 = (1-f)b \cdot Y_1/[X_1 + (1-f)b]$.

The key: the second chunk's input reserve is $X_1 = X + a$, but if there were no fee it would be $X + (1-f)a$. The extra fa makes the pool look richer in X, giving a worse rate for the $X \to Y$ direction. Therefore $\text{out}_1 + \text{out}_2 < \text{Out}_{\text{one}}$. □

Example 10 ($f = 1\%$)**.** $(X, Y) = (1000, 1000)$, $f = 0.01$.

One-shot 200: eff. 198; out $= 198{,}000/1198 \approx 165.28$.

Split 100+100: Eff. 99; $\text{out}_1 = 99{,}000/1099 \approx 90.08$. $X_1 = 1100$, $Y_1 = 1{,}000{,}000/1099 \approx 909.92$. Eff. 99; $\text{out}_2 = 99 \times 909.92/1199 \approx 75.10$. Total ≈ 165.18.

One-shot 165.28 > split 165.18: subadditive (safe). ✓

12 Output-Fee: Superadditive (Exploitable)

Definition 3. Fee rate $f \in (0,1)$. Trader receives $(1-f) \cdot \text{out}_{\text{nf}}$. The fee $f \cdot \text{out}_{\text{nf}}$ stays in the output reserve.

Theorem 4. *With output-fee, GrowLiquidity is superadditive:* $\text{Out}(a+b) < \text{Out}(a) + \text{Out}(b; F_a)$ *for* $a, b > 0$.

The exact gap is:

$$\boxed{\text{Gap} = \frac{(1-f)\, f\, ab\, Y}{(X+a)(X+a+b)}.} \tag{12}$$

Proof. **One-shot:** $\text{Out}_{\text{one}} = (1-f)(a+b)Y/(X+a+b)$.

Split: $\text{out}_1 = (1-f)aY/(X+a)$. $Y_1 = Y(X+fa)/(X+a)$ (fee keeps extra Y in pool). $\text{out}_2 = (1-f)bY(X+fa)/[(X+a)(X+a+b)]$.

$$\text{out}_1 + \text{out}_2 = \frac{(1-f)Y}{(X+a)(X+a+b)}\big[(a+b)(X+a) + fab\big],$$

$$\text{Out}_{\text{one}} = \frac{(1-f)Y}{(X+a)(X+a+b)}\big[(a+b)(X+a)\big].$$

Difference: $(1-f)f \cdot ab \cdot Y/[(X+a)(X+a+b)] > 0$. □

Example 11 ($f = 1\%$)**.** $(X, Y) = (1000, 1000)$, $f = 0.01$.

One-shot 200: fee-free out 166.67; trader gets $0.99 \times 166.67 \approx 165.00$.

Split 100+100: $\text{out}_1 = 0.99 \times 90.91 \approx 90.00$. $Y_1 = 1000 \times (1000+10)/1100 = 1000 \times 1010/1100 \approx 918.18$. $\text{out}_2 = 0.99 \times 100 \times 918.18/1200 \approx 75.75$. Total ≈ 165.75.

Split 165.75 > one-shot 165.00: superadditive (exploitable). ✓

Exact gap: $0.99 \times 0.01 \times 100 \times 100 \times 1000/(1100 \times 1200) = 0.75$.

Part IV
Summary

13 All Issues and Fixes at a Glance

Bug	Problem	Fix	Severity
1	Delta formulas are swapped between tokenA and tokenB	Swap the sqrt ratios (§5)	High
2	3D/4D pairwise decomposition allows triangular arbitrage	Disable 3D/4D (§8)	Critical

14 Additivity Summary

Configuration	Additivity	One-shot vs. split	Safe?
2D, no fee	Strictly additive	Equal	Yes
2D + input-fee	Subadditive	One-shot better	Yes
2D + output-fee	Superadditive	Split better	**No**
3D / 4D (any fee)	Broken: triangular arbitrage drains pool		**No**